

FLOW

Student Study Task Planner

Project Report

Course Code	CSE1021
Course	Introduction to Problem Solving and Programming
Project Type	VITyarthi - Build Your Own Project
Student Name	Sarthak Dhawan
Registration Number	26BAI10618
Branch / Semester	B.Tech / Fall semester
Session	2026-2027
Name of Faculty	Dr. Jitendra P S Mathur
Subject	Introduction to Problem Solving and Programming
Module	1
Sub Code	CSE-1021
Slot	A21+A22+A23+B14+D21+D22

The project uses basic Python programming, data structures, control flow, functions, and an explicit list-ordering algorithm from the CSE1021 syllabus.

1. Introduction

Flow is a command-line study planner for college students. It provides one place to enter study tasks, see their status, generate a simple order for pending work, and check progress. The project is designed around the problem-solving and programming concepts covered in CSE1021 rather than advanced frameworks or external services.

The application follows a top-down design. The main program controls the menu, while separate modules handle task operations, validation, planning, analysis, and sample data. This keeps the program easy to read and test.

2. Problem Statement

Students may have assignments, lab work, revision, and other study activities across several subjects. A plain list records these tasks but does not directly answer which task should be handled first. Flow addresses this small problem by using deadline urgency, academic priority, and estimated effort to produce a ranked study plan.

3. Objectives

1. Create a simple menu-driven student task planner. 2. Apply Python lists and dictionaries to represent task data. 3. Use functions, conditionals, loops, and validation for input processing. 4. Implement a basic priority algorithm and selection sort. 5. Calculate simple progress statistics using counting and sets.

4. Functional Requirements

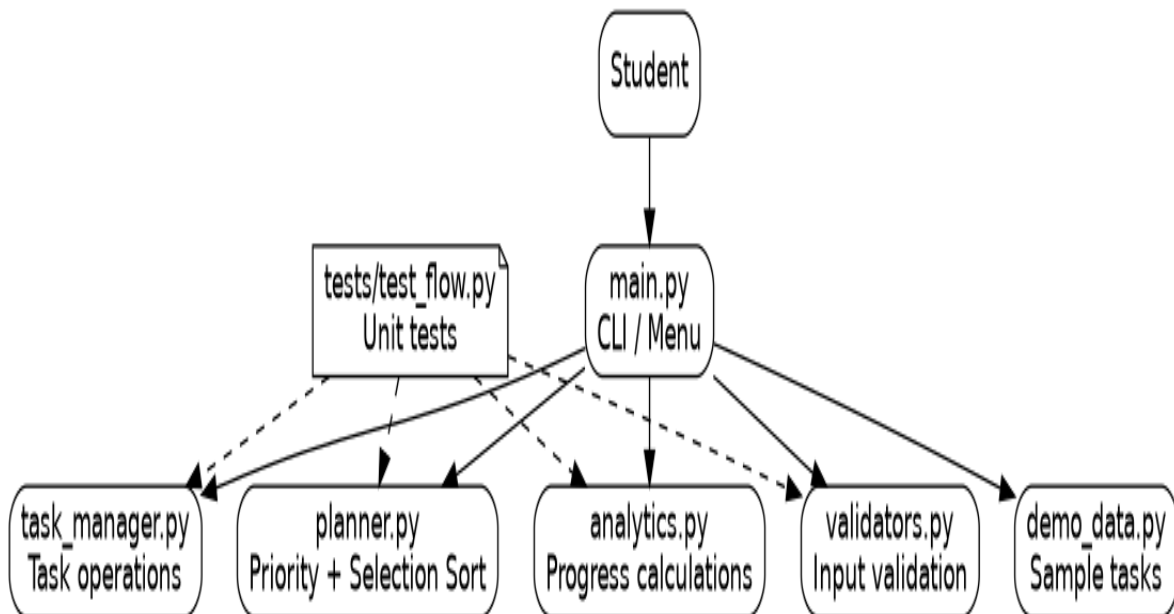
Module	Function	Input	Output
Task Management	Add, view, complete, delete	Task details / task ID	Updated task list or action result
Study Planner	Score and rank pending tasks	Pending tasks	Ordered study plan
Progress Analysis	Calculate completion and subject statistics	Task list	Progress summary

5. Non-Functional Requirements

Requirement	Implementation
Performance	The program uses simple in-memory list operations suitable for a normal student task list.
Usability	The numbered menu, clear prompts, and readable outputs guide the user through each operation.
Reliability	Invalid menu choices, dates, priorities, and numeric values are checked before use.
Maintainability	Major features are divided into small Python modules with focused functions.
Portability	Only the Python standard library is required.

6. System Architecture

The system is a small modular command-line application. The main module receives input and connects the three functional modules. Validation is separated so that user input is checked before being passed to task operations. The test module exercises the reusable functions.



7. Design Diagrams

7.1 Use Case Diagram

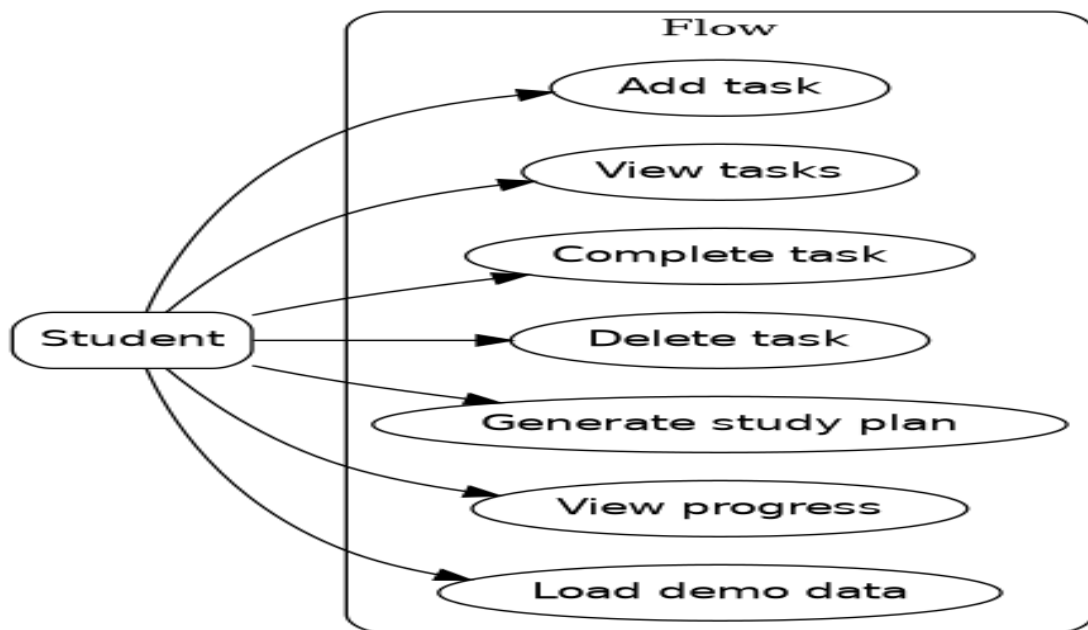


Figure 1. User actions supported by Flow.

7.2 Workflow Diagram

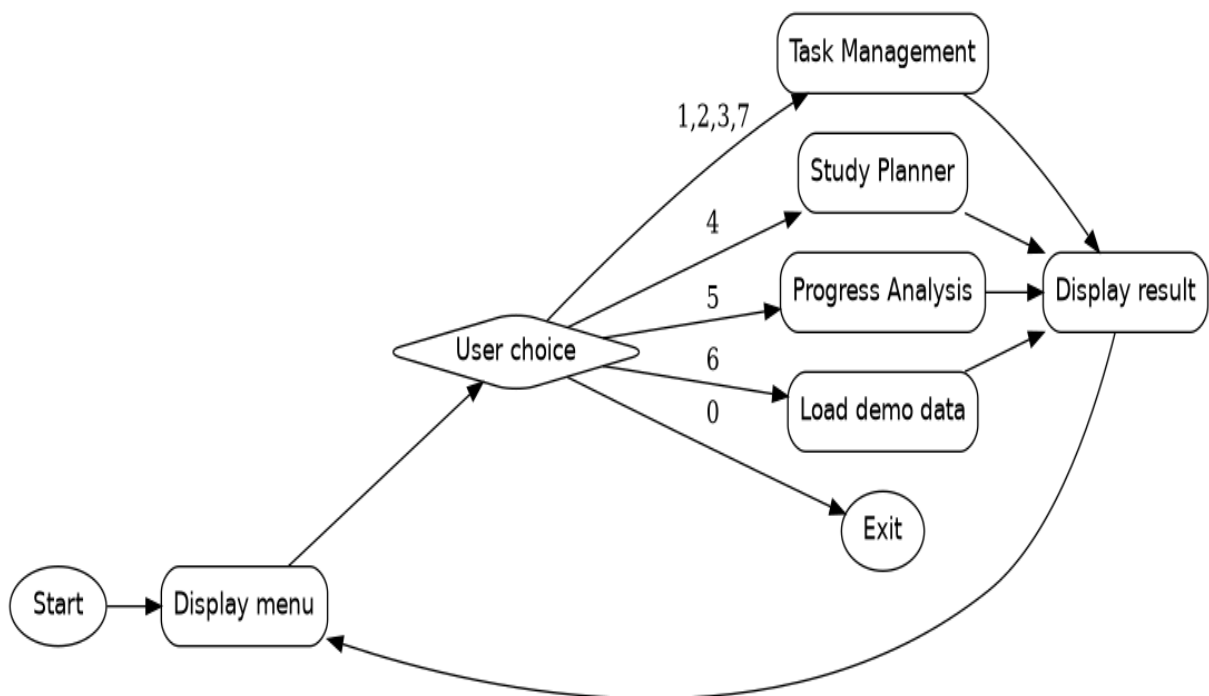


Figure 2. Main program workflow.

7.3 Sequence Diagram

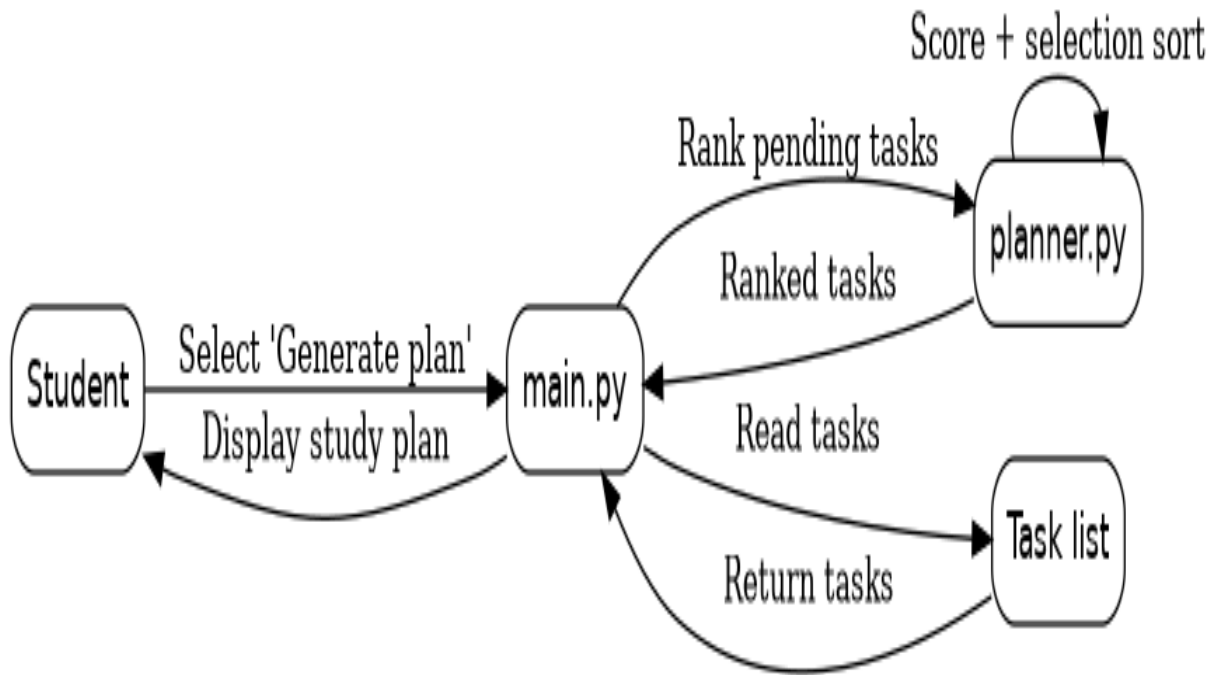


Figure 3. Study-plan generation sequence.

7.4 Component Diagram

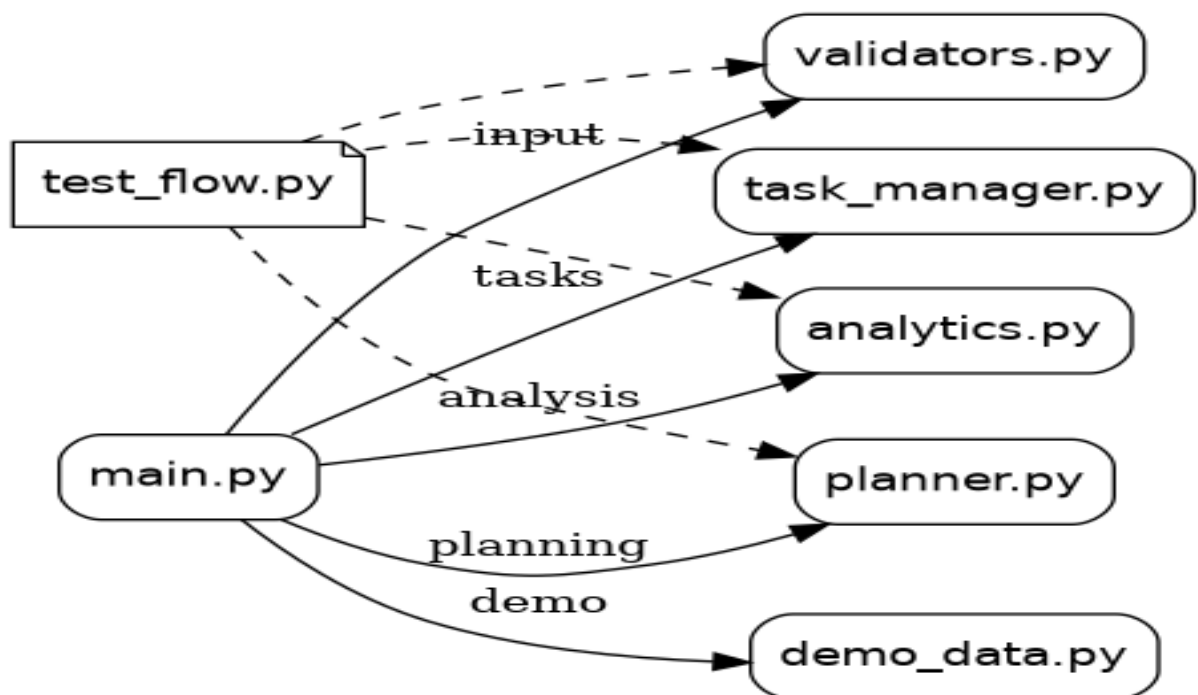


Figure 4. Program modules and test dependency.

ER Diagram: Not applicable because this version does not use persistent database storage.

8. Design Decisions and Rationale

The project deliberately uses only the concepts needed for CSE1021. Tasks are represented as dictionaries inside a list. This directly demonstrates Python list and dictionary usage. A set is used to count unique subjects, and a dictionary is used for subject-wise counts.

The study planner uses a deterministic weighted score. It is not a machine-learning model and does not use an external API. The score is calculated from three values so that the algorithm can be explained during evaluation. Selection sort is implemented manually to demonstrate an elementary array/list technique from the course.

No database, GUI, web framework, authentication system, or cloud service is included. This keeps the project within the course level while still satisfying the project requirements.

9. Implementation Details

The source code is divided into the following modules:

main.py - controls the CLI menu and connects modules.

task_manager.py - adds, displays, completes, and deletes tasks.

planner.py - calculates urgency, score, explanation, and selection sort.

analytics.py - calculates completion percentage and subject statistics.

validators.py - validates text, dates, menu choices, priority, and numeric input.

demo_data.py - provides sample tasks for evaluation.

tests/test_flow.py - checks core functions using unittest.

Study-plan score:

```
Score = 0.50 x Urgency + 0.30 x Academic Priority + 0.20 x Effort Score
```

10. Screenshots / Results

A demo run loads four sample tasks, generates a ranked plan, displays progress, and lists the tasks. The output is produced directly in the terminal.

Flow CLI Demo

```
=====
FLOW - STUDENT STUDY TASK PLANNER
A simple Python project for managing college study work
=====

MENU
1. Add task
2. View tasks
3. Mark task complete
4. Generate study plan
5. View progress
6. Load demo data
7. Delete task
0. Exit
Choose an option: Demo data loaded.

MENU
1. Add task
2. View tasks
3. Mark task complete
4. Generate study plan
5. View progress
6. Load demo data
7. Delete task
0. Exit
Choose an option:
STUDY PLAN
-----
1. Python lab | Score: 91.0
   Programming | Due: 2026-09-22 | Priority: 5/5 | 2.0h
```

Figure 5. Sample terminal execution.

11. Testing Approach

The project uses Python's built-in **unittest** module. Five tests cover task creation, completion and deletion, priority scoring, selection sort, analytics, and date validation.

Observed result: **Ran 5 tests ... OK.**

12. Challenges Faced

The main challenge was dividing the problem into small steps without making the program unnecessarily complex. Another challenge was designing a priority rule that is simple enough to explain but still useful. Input validation was also required so that incorrect menu values or task details do not stop the program.

13. Learnings and Key Takeaways

The project reinforced top-down problem solving, modular functions, conditionals, loops, list and dictionary operations, sets, input validation, and basic algorithm design. Implementing selection sort instead of using a built-in sorting function also helped connect the project to the algorithm topics in the syllabus.

14. Future Enhancements

A future version could save tasks between runs or add a graphical interface. These features are intentionally outside the present version so that the current implementation remains focused on the CSE1021 programming syllabus.

15. References

1. VITyarthi, Build Your Own Project - General Project Instructions and Submission Guidelines, provided course document.
2. CSE1021, Introduction to Problem Solving and Programming, course outline and syllabus provided for the project.

Appendix: Run and Test Commands

Run:

```
python main.py
```

Tests:

```
python -m unittest discover -s tests -v
```